



DOSSIER DE CONCEPTION

Fonctionnelle et Technique

Système Intelligent de Gestion de Bibliothèque

Projet	Système Intelligent de Gestion de Bibliothèque (Library Management System)
Version	1.0.0
Date	Juin 2026
Stack technique	Java 21, Spring Boot 4.0.6, Spring Data JPA, Spring Security, JWT, MySQL 8, PostgreSQL, Maven, Lombok
Porteur	Équipe ProjSpring



Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Objectifs du document	3
1.3	Périmètre	3
2	Présentation du projet	3
2.1	Description générale	3
2.2	Problématique	4
2.3	Objectifs	4
3	Analyse fonctionnelle	5
3.1	Acteurs du système	5
3.2	Fonctionnalités principales	5
4	Modélisation de la base de données	6
4.1	Les tables	6
4.1.1	Liste des entités	6
4.1.2	Schéma des entités principales	6
4.2	MLD (Modèle Logique de Données)	6
4.3	Diagramme de cas d'utilisation	7
4.4	Diagramme de séquence général – Emprunt d'un livre	7
5	Architecture générale du système	8
5.1	Vue d'ensemble	8
5.2	Flux de communication	8
5.3	Déploiement	8
6	APIs principales (REST)	9
6.1	Endpoints applications	9
6.2	Exemples JSON	9
6.2.1	Création d'un livre – Requête	9
6.2.2	Réponse	10
6.2.3	Emprunt d'un livre	10
6.2.4	Authentification	10
7	Architecture technique	12
7.1	Structure du projet Spring Boot	12
7.1.1	Détails par couche	12
7.2	Technologies et dépendances	12
8	Planification du projet	13
8.1	Découpage en phases	13
8.2	Planning détaillé sur 4 semaines	13
8.3	Répartition des responsabilités	13



9 Conclusion	14
9.1 Synthèse	14
9.2 Récapitulatif	14



1 Introduction

1.1 Contexte

La gestion manuelle des bibliothèques présente des limites considérables : perte de traces des emprunts, difficulté à localiser les ouvrages, files d'attente non structurées et absence de visibilité en temps réel sur l'état du fonds documentaire. Le présent projet vise à répondre à ces défis par la conception et le développement d'un système informatique complet de gestion de bibliothèque.

Ce projet s'inscrit dans le cadre d'une initiative visant à moderniser les infrastructures documentaires en proposant une solution open-source, modulaire et extensible.

1.2 Objectifs du document

Ce document a pour objectifs de :

- Présenter l'analyse fonctionnelle et technique du système ;
- Décrire l'architecture et la modélisation de la base de données ;
- Détailler les APIs REST exposées ;
- Planifier les phases de développement et la répartition des tâches.

1.3 Périmètre

Volet	Inclus	Exclus
Fonctionnel	Gestion des livres, adhérents, emprunts, réservations	Marketing, vente d'ouvrages
Technique	API REST, JWT, base de données relationnelle, Docker	Application mobile native, IA
Utilisateurs	Bibliothécaires, administrateurs, assistants	Usagers finaux en self-service

2 Présentation du projet

2.1 Description générale

Vision du projet

Développer une API REST modulaire, sécurisée et performante pour la gestion intelligente des bibliothèques, capable de gérer le catalogue documentaire, les emprunts, les réservations et les adhérents, tout en respectant les règles métier et



en offrant une extensibilité maximale.

2.2 Problématique

Les bibliothèques traditionnelles font face à plusieurs difficultés :

- **Gestion manuelle** des emprunts et retards, source d'erreurs et d'inefficacité ;
- **Absence de réservation structurée** : les membres ne peuvent pas réserver un ouvrage indisponible ;
- **Quotas non contrôlés** : aucun suivi du nombre d'emprunts par adhérent ;
- **Traçabilité limitée** : impossible de connaître l'historique complet d'un exemplaire ;
- **Sécurité insuffisante** : accès non contrôlé aux données sensibles.

2.3 Objectifs

N°	Objectif
1	Automatiser le cycle d'emprunt et de retour des ouvrages
2	Mettre en place un système de réservation FIFO avec file d'attente
3	Appliquer des règles métier (quotas, prolongations, délais)
4	Sécuriser l'accès par authentification JWT et rôles
5	Assurer l'intégrité des données via le soft-delete et l'optimistic locking
6	Exposer une API REST documentée et testable

3 Analyse fonctionnelle

3.1 Acteurs du système

Acteur	Description	Rôle
ADMIN	Administrateur système avec tous les droits	Gère les utilisateurs, la configuration, les données de référence
LIBRARIAN	Bibliothécaire habilité	Gère les livres, adhérents, emprunts et réservations
ASSISTANT	Assistant de bibliothèque	Effectue les opérations courantes (emprunt, retour)
USER	Consultant simple	Consulte le catalogue et son historique

3.2 Fonctionnalités principales

N°	Module	Description	Acteurs
1	Gestion du catalogue	CRUD livres, auteurs, catégories, éditeurs, langues	ADMIN, LIBRARIAN
2	Gestion des exemplaires	Suivi des exemplaires physiques par code-barres et statut	ADMIN, LIBRARIAN
3	Gestion des adhérents	Inscription, modification, quotas par type (STUDENT/TEACHER/EXTERNAL)	ADMIN, LIBRARIAN
4	Prêt / Retour	Emprunt avec vérification de disponibilité et quota, retour, prolongation	LIBRARIAN, ASSISTANT
5	Réservation	File d'attente FIFO, annulation, consultation de la file	LIBRARIAN, ASSISTANT
6	Authentification	Login JWT, gestion de profil, sécurisation des routes	Tous
7	Administration	Gestion des utilisateurs, seed des références	ADMIN

4 Modélisation de la base de données

4.1 Les tables

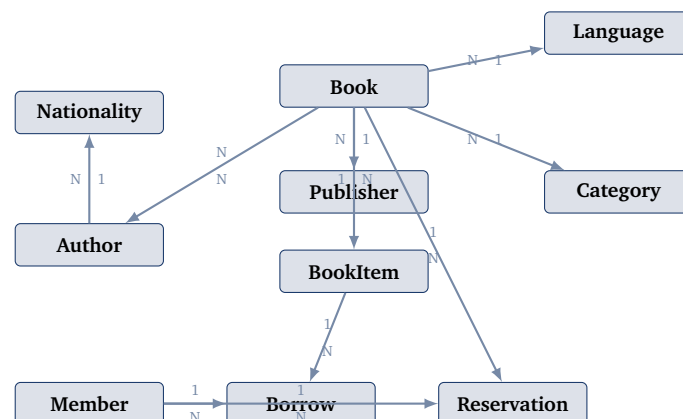
4.1.1 Liste des entités

Le système repose sur **11 entités JPA** et **5 énums** :

N°	Entité	Description	Soft delete
1	Language	Langue (code ISO + nom)	Non
2	Nationality	Nationalité (code ISO + nom)	Non
3	Category	Catégorie thématique de livre	Oui
4	Publisher	Maison d'édition	Oui
5	Author	Auteur lié à une nationalité	Oui
6	Book	Œuvre intellectuelle (ISBN unique)	Oui
7	BookItem	Exemplaire physique (code-barres unique, version locking)	Oui
8	BookAuthor	Relation N-N livre-auteur avec rôle	N/A
9	Member	Adhérent (email unique, type, quota)	Oui
10	Borrow	Emprunt (membre, exemplaire, dates, statut)	Non
11	Reservation	Réservation (membre, livre, position file)	Non
12	User	Utilisateur (auth)	N/A

Enums : AuthorRole (MAIN_AUTHOR, CO_AUTHOR, EDITOR), BookItemStatus (AVAILABLE, BORROWED, RESERVED, LOST, DAMAGED), BorrowStatus (ACTIVE, RETURNED, OVERDUE, LOST), MemberType (STUDENT, TEACHER, EXTERNAL), ReservationStatus (PENDING, READY, CANCELLED, COMPLETED).

4.1.2 Schéma des entités principales

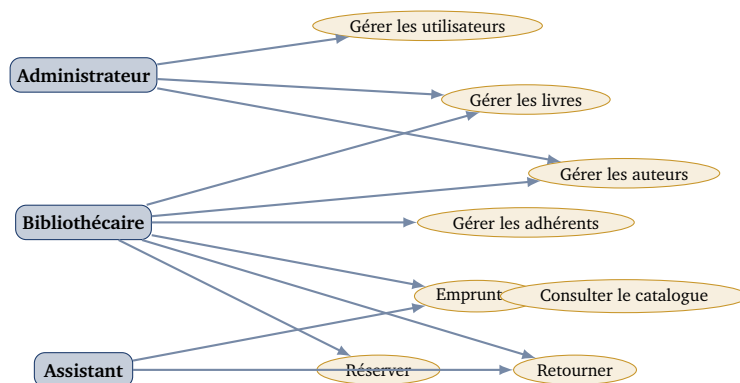


4.2 MLD (Modèle Logique de Données)

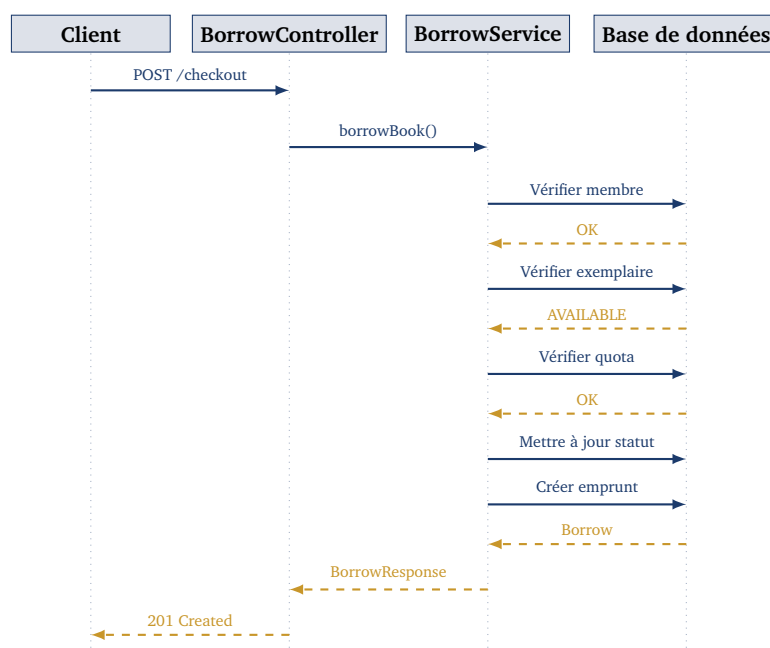
Le schéma relationnel ci-dessous décrit les tables et leurs relations :

Table	Clé primaire	Clés étrangères
language	code (PK)	–
nationality	code (PK)	–
category	id (PK)	–
publisher	id (PK)	–
author	id (PK)	nationality_code → nationality
book	id (PK)	language_code → language, category_id → category, publisher_id → publisher
book_item	id (PK)	book_id → book
book_author	(book_id, author_id) (PK)	book_id → book, author_id → author
member	id (PK)	–
borrow	id (PK)	member_id → member, book_item_id → book_item
reservation	id (PK)	member_id → member, book_id → book
library_user	id (PK)	–

4.3 Diagramme de cas d'utilisation



4.4 Diagramme de séquence général – Emprunt d'un livre

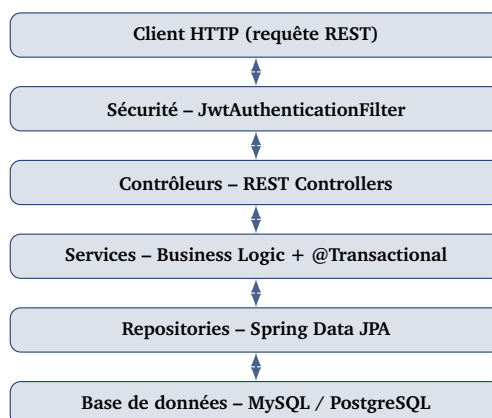


5 Architecture générale du système

5.1 Vue d'ensemble

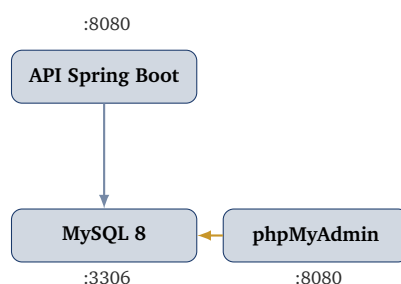
Couche Frontend	Couche Backend (API)	Base de données
Client HTTP (Postman, cURL, navigateur)	Java 21 + Spring Boot 4.0.6	MySQL 8 (dev) / PostgreSQL (prod)
	Contrôleurs REST	H2 (tests)
	Services métier	spring-jpa + Hibernate
	Sécurité JWT	

5.2 Flux de communication



5.3 Déploiement

Le système est déployé via Docker avec l'architecture suivante :



Le Dockerfile utilise une construction multi-étapes :

- Étape 1 : compilation avec maven:3.9-eclipse-temurin-21
- Étape 2 : exécution avec eclipse-temurin:21-jre (image finale ~180MB)
- Profil actif : prod avec PostgreSQL

6 APIs principales (REST)

6.1 Endpoints applications

Méthode	Endpoint	Contrôleur	Description
POST	/api/auth/login	AuthController	Authentification JWT
PUT	/api/auth/profile	AuthController	Mise à jour identifiants
GET	/api/books	BookController	Liste paginée des livres
POST	/api/books	BookController	Création d'un livre
GET	/api/books/{id}	BookController	Détail d'un livre
PUT	/api/books/{id}	BookController	Modification d'un livre
DELETE	/api/books/{id}	BookController	Suppression logique
GET	/api/authors	AuthorController	Liste paginée des auteurs
POST	/api/authors	AuthorController	Création d'un auteur
PUT	/api/authors/{id}	AuthorController	Modification d'un auteur
DELETE	/api/authors/{id}	AuthorController	Suppression logique
GET	/api/books/{id}/i	BookItemController	Exemplaires d'un livre
POST	/api/books/{id}/i	BookItemController	Ajout d'un exemplaire
GET	/api/books/{id}/a	BookAuthorController	Auteurs d'un livre
POST	/api/books/{id}/a	BookAuthorController	Associer un auteur
GET	/api/members	MemberController	Liste paginée des membres
POST	/api/members	MemberController	Création d'un membre
GET	/api/borrows	BorrowController	Liste paginée des emprunts
POST	/api/borrows/check	BorrowController	Emprunter un livre
POST	/api/borrows/{id}	BorrowController	Retourner un livre
POST	/api/borrows/{id}	BorrowController	Prolonger un emprunt
GET	/api/reservations	ReservationController	Liste paginée
POST	/api/reservations	ReservationController	Réserver un livre
POST	/api/reservations	ReservationController	Annuler une réservation
GET	/api/reservations	ReservationController	File d'attente
GET	/api/categories	CategoryController	Liste des catégories
GET	/api/publishers	PublisherController	Liste des éditeurs
GET	/api/languages	LanguageController	Liste des langues
GET	/api/nationalities	NationalityController	Liste des nationalités
GET	/api/admin/users	AdminController	Utilisateurs (admin)
POST	/api/admin/users	AdminController	Créer un utilisateur
POST	/api/admin/seed	SeedController	Initialiser les références

6.2 Exemples JSON

6.2.1 Création d'un livre – Requête

```
1 POST /api/books
2 Authorization: Bearer <token>
3 Content-Type: application/json
4
```



```
5 {
6   "title": "Le Petit Prince",
7   "isbn": "978-2-07-040850-4",
8   "languageCode": "FR",
9   "categoryId": 1,
10  "publisherId": 1
11 }
```

6.2.2 Réponse

```
1 HTTP/1.1 201 Created
2
3 {
4   "id": 1,
5   "title": "Le Petit Prince",
6   "isbn": "978-2-07-040850-4",
7   "languageCode": "FR",
8   "languageName": "French",
9   "categoryId": 1,
10  "categoryName": "Litterature",
11  "publisherId": 1,
12  "publisherName": "Gallimard"
13 }
```

6.2.3 Emprunt d'un livre

```
1 POST /api/borrows/checkout?memberId=1&barcode=BK-001
2
3 HTTP/1.1 201 Created
4
5 {
6   "id": 1,
7   "memberId": 1,
8   "memberEmail": "jean@example.com",
9   "bookItemId": 1,
10  "bookItemBarcode": "BK-001",
11  "renewalCount": 0,
12  "borrowDate": "2026-06-05T10:30:00",
13  "dueDate": "2026-06-19",
14  "returnDate": null,
15  "status": "ACTIVE"
16 }
```

6.2.4 Authentication

```
1 POST /api/auth/login
2 Content-Type: application/json
3
```



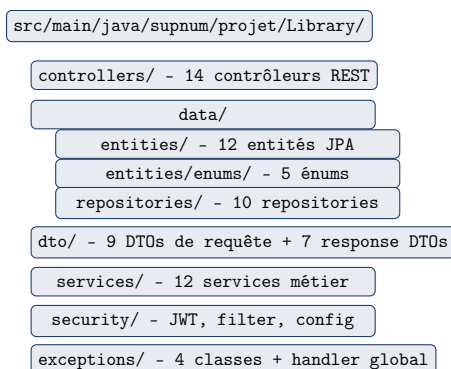
```
4 {  
5   "username": "admin",  
6   "password": "password"  
7 }  
8  
9 HTTP/1.1 200 OK  
10  
11 {  
12   "token": "eyJhbGciOiJIUzI1NiJ9...",  
13   "username": "admin",  
14   "role": "ADMIN"  
15 }
```



7 Architecture technique

7.1 Structure du projet Spring Boot

L'arborescence du projet suit une architecture en couches classique :



7.1.1 Détails par couche

Couche	Responsabilité
Controllers	Endpoints REST, validation (@Valid), mapping DTO ↔ response
Services	Logique métier, transactions (@Transactional), règles de gestion
Repositories	Spring Data JPA, requêtes personnalisées, pagination
Entities	Mapping JPA, relations, soft delete, optimistic locking
Security	Filtre JWT, encodage BCrypt, configuration Spring Security
Exceptions	Gestion centralisée des erreurs (404, 400, 409, 500)

7.2 Technologies et dépendances

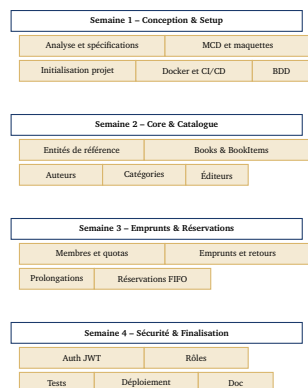
Catégorie	Technologie	Version
Langage	Java (JDK)	21 LTS
Framework	Spring Boot	4.0.6
ORM	Spring Data JPA / Hibernate	(géré par Boot)
Sécurité	Spring Security + JWT (jjwt)	0.12.6
Validation	Jakarta Validation (Hibernate Validator)	(géré par Boot)
Base de données	MySQL (dev), PostgreSQL (prod), H2 (test)	8 / 16 / 2
Build	Maven Wrapper	3.9.15
Utilitaires	Lombok, DevTools	(géré par Boot)
Conteneurisation	Docker + Docker Compose	24+
Déploiement	Render (Cloud)	–

8 Planification du projet

8.1 Découpage en phases

Phase	Description	Livrables	Durée
1	Conception	Spécifications, MCD, maquettes	1 semaine
2	Setup	Init projet, CI/CD, Docker, BDD	3 jours
3	Core (entités de référence)	Language, Nationality, Category, Publisher, Author	4 jours
4	Catalogue	Book, BookItem, BookAuthor	5 jours
5	Membres & Emprunts	Member, Borrow, règles métier	5 jours
6	Réservations	Reservation, file FIFO, annulation	4 jours
7	Sécurité	Auth JWT, rôles, filtres	3 jours
8	Tests & Déploiement	Tests unitaires/intégration, docker, déploiement	4 jours

8.2 Planning détaillé sur 4 semaines



8.3 Répartition des responsabilités

Membre	Modules	Tâches
Membre 1	Security, BaseEntity, Exceptions	Configuration JWT, filtres, GlobalExceptionHandler, soft delete
Membre 2	Références	Language, Nationality, Category, Publisher, Author (CRUD)
Membre 3	Catalogue	Book, BookItem (optimistic locking), BookAuthor
Membre 4	Membres & Emprunts	Member, Borrow (quotas, prolongations, retours)
Membre 5	Réservations	Reservation (file FIFO, annulation, queue)



9 Conclusion

Le présent dossier de conception a permis de formaliser l'ensemble des spécifications fonctionnelles et techniques du **Système Intelligent de Gestion de Bibliothèque**.

9.1 Synthèse

Le système proposé repose sur une architecture modulaire en couches, offrant :

- Une **API REST complète** couvrant l'ensemble du cycle de vie documentaire ;
- **11 entités JPA** et **5 énums** pour une modélisation exhaustive du domaine ;
- **14 contrôleurs** et **12 services** encapsulant la logique métier ;
- Une **sécurité robuste** via JWT avec gestion des rôles ;
- Des **mécanismes avancés** : soft delete, optimistic locking, file d'attente FIFO ;
- Un **déploiement conteneurisé** avec Docker et PostgreSQL.

9.2 Récapitulatif

Axe	Apport du projet
Modernisation	Automatisation complète de la gestion documentaire
Performance	API REST rapide avec pagination et cache JPA
Sécurité	Authentification JWT, rôles, validation des entrées
Extensibilité	Architecture modulaire, prête pour l'ajout de fonctionnalités
Maintenabilité	Code organisé en couches, tests H2, documentation

ProjSpring

Système Intelligent de Gestion de Bibliothèque
